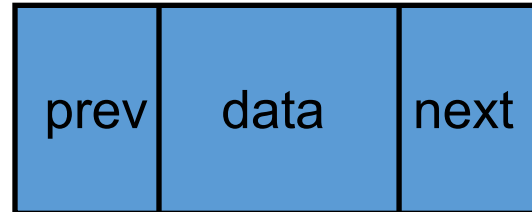


Data Structures and Algorithms (CS221)

Double and Circular Linked List

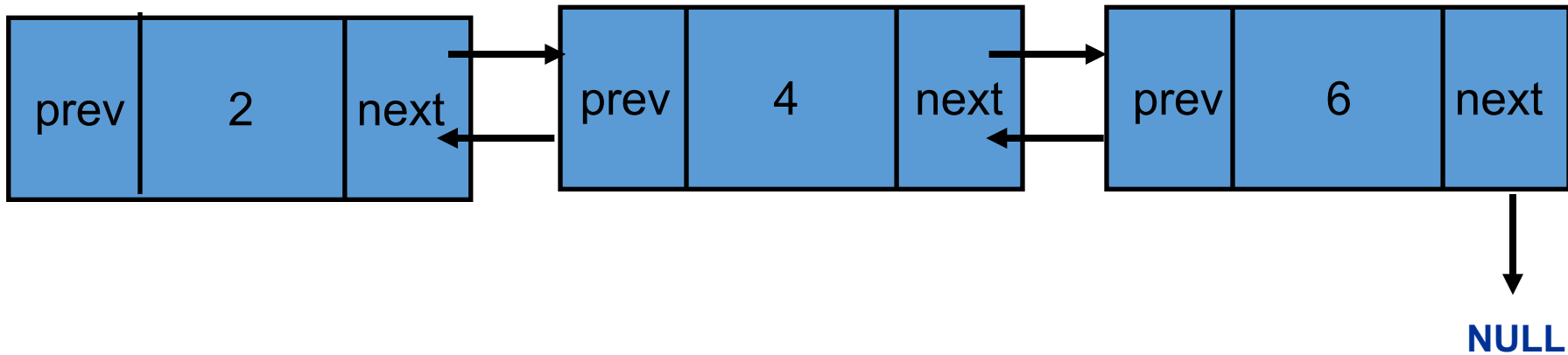
Double Linked List

node



head

tail



Double Linked List: Create the head node

```
struct node
{
    int data;
    node *next;
    node *prev;
};
node *head, *tail;
void main()
{
    head = new node;
    head->next = NULL;
    head->prev = NULL;
    tail = head;
}
```

int data → Stores the actual value.

node *next → A pointer to the **next** node

node *prev → A pointer to the **previous** node.

head → Points to the first node and **tail** → Points to the last node of the list.

new node; → Dynamically allocates memory for a new node

Since this is the **only node**, its next pointer is NULL.

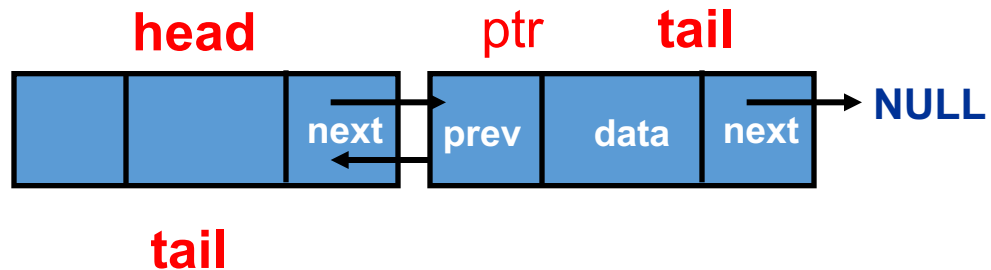
No previous node, so prev is NULL

tail = head; → Since there's only one node, both head and tail point to it.

Double Linked List: Add an element after the head node

```
node *ptr = new node;  
cin >> ptr->data;  
head->next = ptr;  
ptr->prev = head;  
ptr->next = NULL;  
tail = ptr;
```

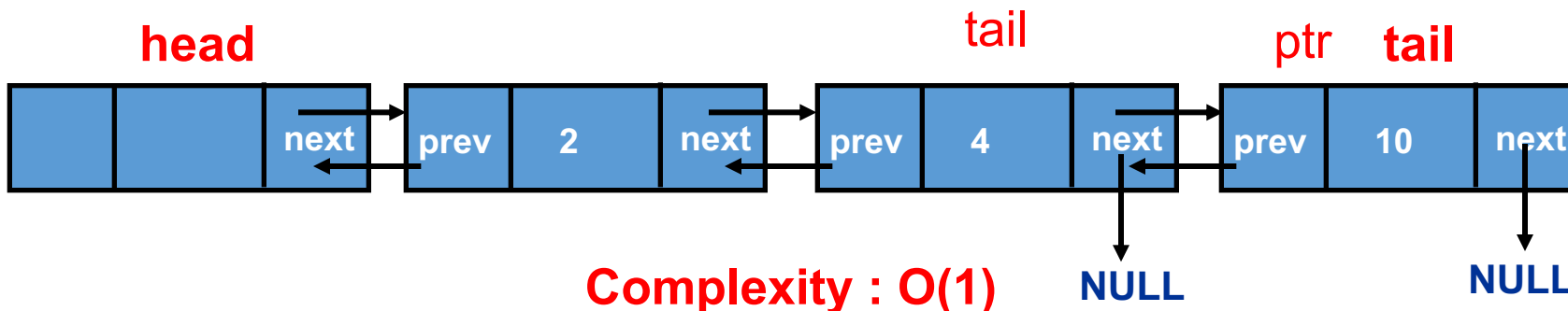
Complexity : $O(1)$



Double Linked List: Append a new element after the tail node

- Suppose you want to insert after the node with data = *'value'*

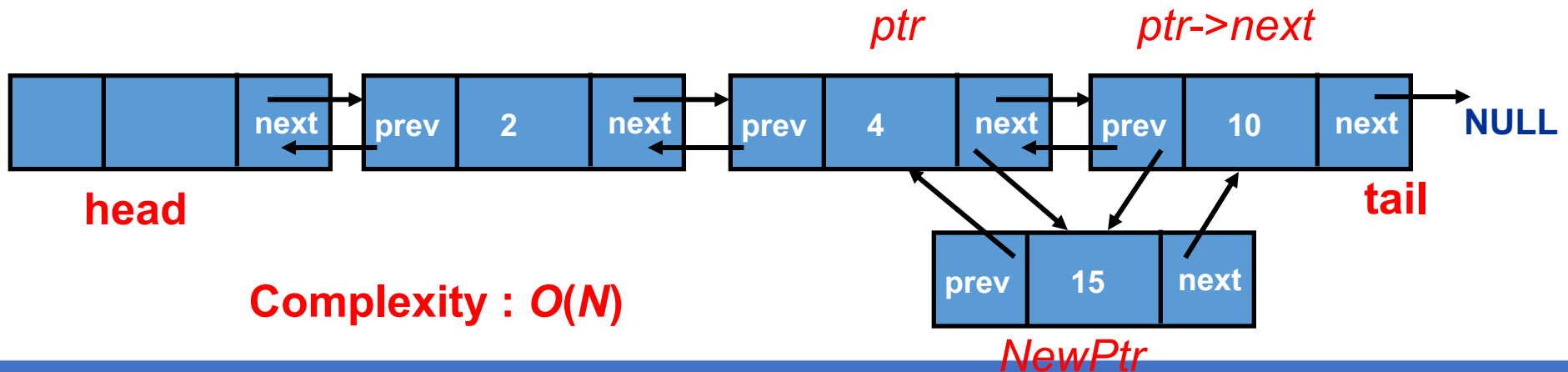
```
ptr = new node;
cin >> ptr->data = // e.g ptr->data = 10;
if (value == tail->data) {
    tail->next = ptr;
    ptr->prev = tail;
    ptr->next = NULL;
    tail = ptr;
}
```



Double Linked List: Insert a new element after a specific node

- Suppose you want to insert after the node with data = *'Aftervalue'*

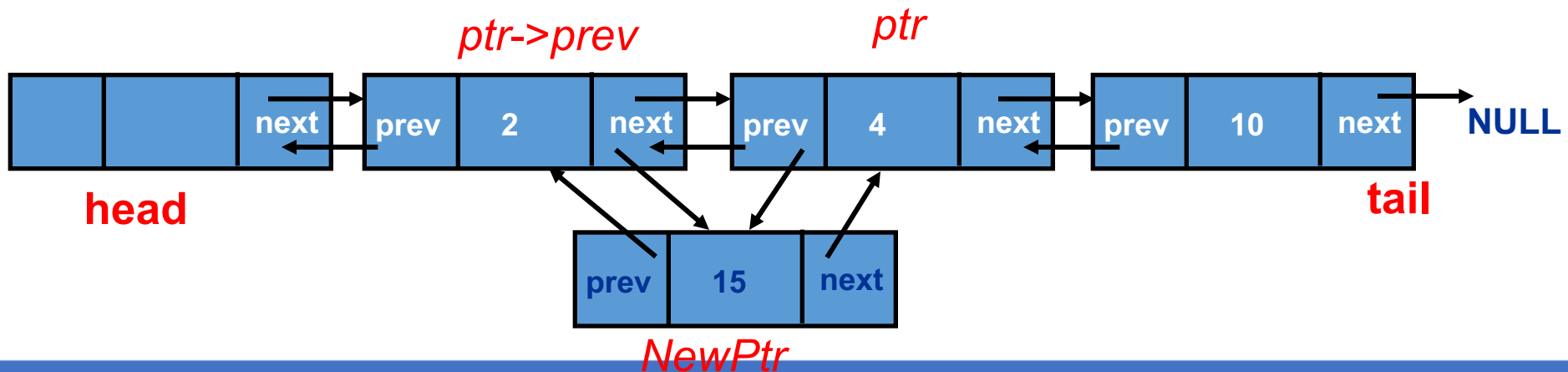
```
node* NewPtr;
for(node *ptr = head->next; ptr != NULL; ptr = ptr->next){
    if(ptr->data == AfterValue) {      e.g AfterValue = 4
        NewPtr = new node;
        NewPtr->data = NewData; //   e.g NewData = 15;
        NewPtr->next = ptr->next;
        (NewPtr->next)->prev = NewPtr;
        ptr->next = NewPtr;
        NewPtr->prev = ptr;
    } // end if
} // end for
```



Double Linked List: Insert a new element before a specific node

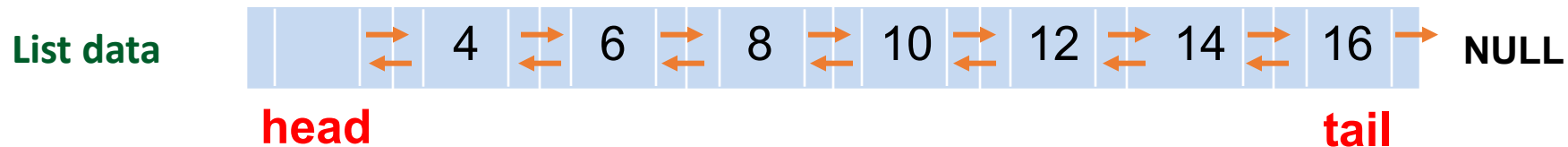
- Suppose you want to insert after the node with data = '*Beforevalue*'

```
node* NewPtr;
for(node *ptr = head->next; ptr != NULL; ptr = ptr->next){
    if(ptr->data == BeforeValue) {      e.g BeforeValue =4
        NewPtr = new node;
        NewPtr->data = NewData; //    e.g NewData =15;
        NewPtr->next = ptr;
        (ptr->prev)->next = NewPtr;
        NewPtr->prev = ptr->prev;
        ptr->prev = NewPtr;
    } // end if
} // end for
```



Double Linked List: Traverse through the linked list

Visit each element of the list
e.g., print each element of the list on
the screen



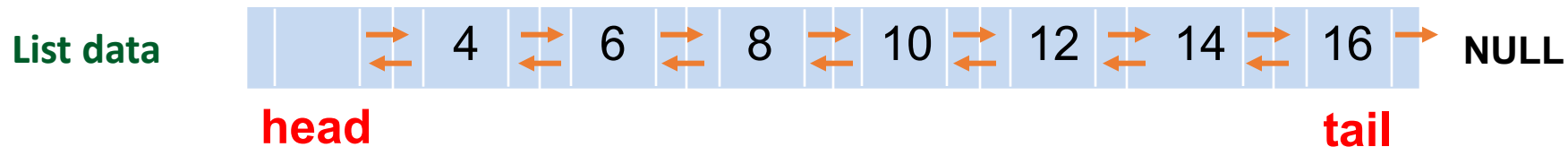
```
for (node *ptr = head->next; ptr != NULL; ptr = ptr->next)
    cout<< ptr->data<<" ";
```

OUTPUT: 4 6 8 10 12 14 16

Complexity : $O(N)$

Double Linked List: Traverse through the linked list in the reverse order

- Visit each element of the list, starting for the last element, in the reverse order
 - e.g., print each element of the list, in the reverse order, on the screen



```
for (node *ptr = tail; ptr != head; ptr = ptr->prev)
    cout<< ptr->data<<" ";
```

OUTPUT: 16 14 12 10 8 6 4

Complexity : $O(N)$

Question



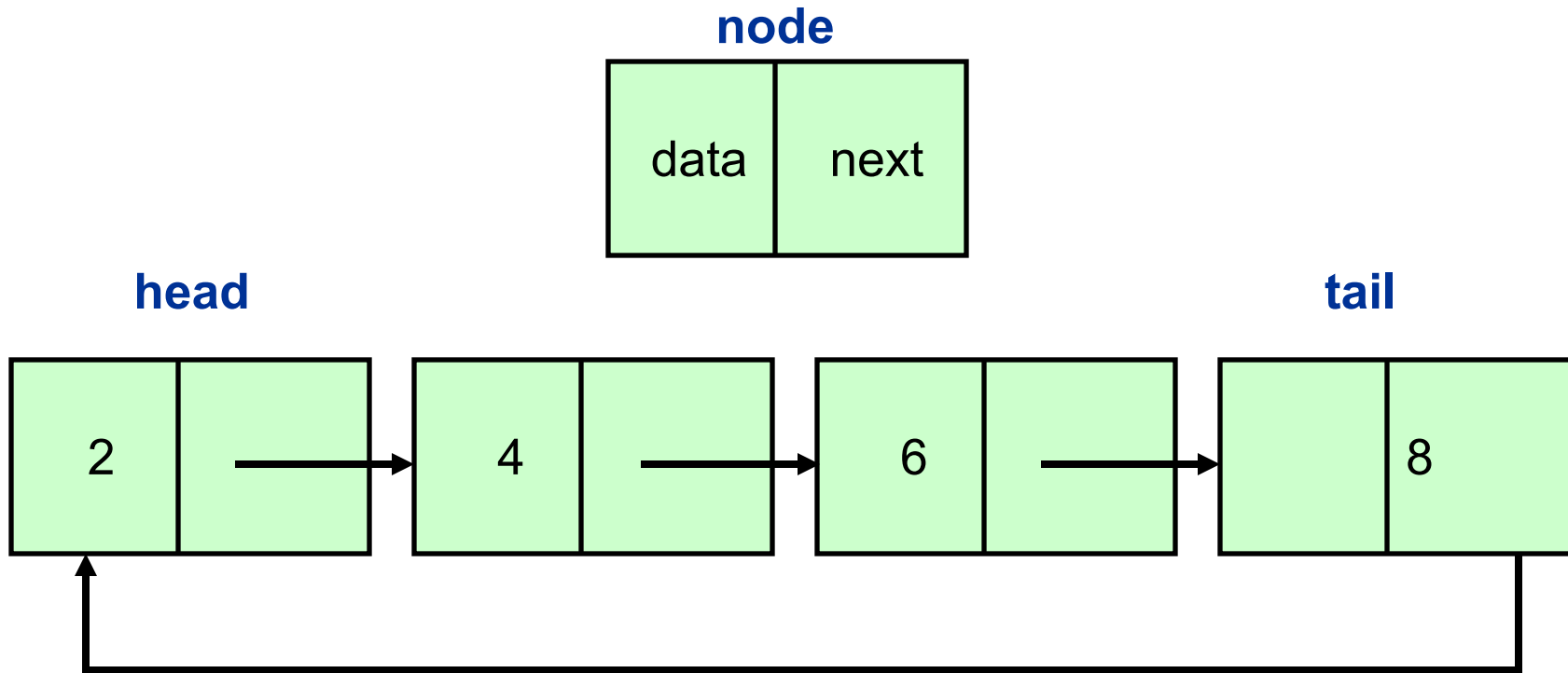
- Can we traverse/print the elements of the a simple linked list (not the double linked) in the reverse order?

Circular Linked List

A **circular linked list (CLL)** is a variation of a linked list where:

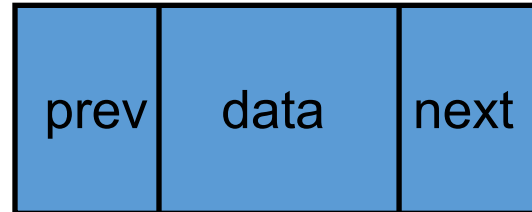
- The last node points back to the first node, forming a circular structure.
- Unlike a normal linked list (which ends in NULL), a circular linked list never ends.

Circler Single Linked List



Circular Doubly Linked List

node



head

tail



Circular Linked List

```
struct node {  
    int data;  
    node *next;  
    node *prev;  
};
```

```
node *head, *tail;
```

```
void main() {  
    head = new node;  
    head->data = 10;  
    head->next = head;  
    head->prev = head;  
    tail = head;  
}
```

DSA Semester Project:

- Online Form (Includes your group members name, project Title, GitHub repo and more details.
- Semester Project File

Summary



- Comparisons of
 - array implementation of static lists
 - array implementation of dynamic lists (dynamic array)
 - single linked list
 - double linked list
 - circular linked list

Summary

- Array implementation of static list vs dynamic list

Static array	Dynamic array
Need to know the memory requirements/ size of the list data in advance	Memory requirements/ size of the list can be adjusted at runtime
Very easy to handle and assign memory (at the beginning)	Complicated runtime memory reallocation procedure
Memory assigned in a single chunk at compile time	Memory assigned in multiple chunks at runtime
Insert/delete operations in the worst case scenario require moving/shifting the whole array elements	Insert/delete operations in the worst case scenario require moving/shifting the whole array elements

Summary

- Dynamic array vs dynamic linked list

Dynamic array	Dynamic linked list
Memory requirements/ size of the list can be adjusted at runtime	Memory requirements/ size of the list can be adjusted at runtime
<u>Memory allocation may not be efficient:</u> memory assigned in multiple chunks at runtime	<u>Memory allocation is more efficient:</u> memory assigned at runtime, one node at a time
<u>Require contiguous memory allocation:</u> frequent insertion/deletion operations of large size may cause <u>fragmentation problem</u>	<u>Effectively addresses the fragmentation problem:</u> the allocated memory is scattered around the available memory space(RAM)
<u>Complicated runtime memory reallocation procedure :</u> need to copy previous data into new memory for each memory re-allocation	<u>Simple runtime memory reallocation procedure:</u> Only the new node is assigned new memory.
<u>Complicated Insert/delete procedures:</u> Insert/delete operations in the worst case scenario require moving/shifting the whole array elements	<u>Simple Insert/delete procedures:</u> Insert/delete operations only require the knowledge/address of one/two nodes
Allow constant-time random access	• Allow only sequential access to elements • Singly linked lists can only be traversed in one direction. • This makes linked lists unsuitable for applications where it's useful to look up an element by its index quickly

Summary

- Single linked list vs Double linked list

Single linked list	Double linked list
Allow only sequential access to elements	• Allow only sequential access to elements
Singly linked lists can only be traversed in one direction.	Doubly linked lists can be traversed in both directions
Insert/delete operations only require the knowledge/address of one/two nodes	Insert/delete operations require the address of only one node: i.e. the node after/before to insert OR node to be deleted

Summary

- simple linked list vs circular linked list



Simple linked list	circular linked list
Requires a NULL pointer implementation (The last nodes points to the NULL)	<u>Simple implementation</u> : no NULL pointers at the ends; no need to check whether pointers are NULL.
<u>The program may crash</u> : due to the presence of NULL pointer, a reference to the NULL pointer is possible	<u>Safe programming practice</u> : a node always has a non-NULL predecessor and successor, and this means that you can always safely dereference its previous and next pointers
<u>May require special handling/different implementation</u> of add-head, add-tail, insert-before and insert-after functions	<u>Simple Implementation</u> : all of add-head, add-tail, insert-before and insert-after functions, can be implemented through a single function
<u>May require special handling/different implementation</u> of delete-head, delete-tail, delete-anywhere functions	<u>Simple Implementation</u> : All of delete-head, delete-tail, delete-anywhere functions, can be implemented through a single function
Merging /Splitting operations of the linked lists are not so simple	<u>A circular list can be split into two circular lists, in constant time</u> , by giving the addresses of the last node of each piece. • The operation consists in swapping the contents of the link fields of those two nodes. • Applying the same operation to any two nodes in two distinct lists joins the two list into one

Questions?

zahmaad.github.io